

12. Design a C program to simulate the concept of Dining-Philosophers Problem

AIM :

To design a C program to simulate the concept of Dining-Philosophers problem

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define NUM_PHILOSOPHERS 5
```

```
#define NUM_CYCLES 5
```

```
pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
```

```
void* philosopherLifeCycle(void *arg)
```

```
{
```

```
    int id = *(int *)arg;
```

```
    int left = id;
```

```
    int right = (id + 1) % NUM_PHILOSOPHERS;
```

```
    for (int i = 0; i < NUM_CYCLES; i++)
```

```
    {
```

```
        // Thinking
```

```
        printf("Philosopher %d is thinking...\n", id);
```

```
        sleep(1);
```

```

// Deadlock prevention
if (id == NUM_PHILOSOPHERS - 1)
{
    pthread_mutex_lock(&chopsticks[right]);
    pthread_mutex_lock(&chopsticks[left]);
}
else
{
    pthread_mutex_lock(&chopsticks[left]);
    pthread_mutex_lock(&chopsticks[right]);
}

// Eating
printf("Philosopher %d is eating...\n", id);
sleep(1);

// Put down chopsticks
pthread_mutex_unlock(&chopsticks[left]);
pthread_mutex_unlock(&chopsticks[right]);
}

printf("Philosopher %d has finished.\n", id);
pthread_exit(NULL);
}

int main()
{

```

```
pthread_t philosophers[NUM_PHILOSOPHERS];  
int philosopher_ids[NUM_PHILOSOPHERS];
```

```
// Initialize mutexes
```

```
for (int i = 0; i < NUM_PHILOSOPHERS; i++)  
{  
    pthread_mutex_init(&chopsticks[i], NULL);  
}
```

```
// Create philosopher threads
```

```
for (int i = 0; i < NUM_PHILOSOPHERS; i++)  
{  
    philosopher_ids[i] = i;  
    pthread_create(&philosophers[i], NULL,  
        philosopherLifeCycle,  
        &philosopher_ids[i]);  
}
```

```
// Wait for all philosophers to finish
```

```
for (int i = 0; i < NUM_PHILOSOPHERS; i++)  
{  
    pthread_join(philosophers[i], NULL);  
}
```

```
// Destroy mutexes
```

```
for (int i = 0; i < NUM_PHILOSOPHERS; i++)  
{
```

```
        pthread_mutex_destroy(&chopsticks[i]);  
    }  
  
    printf("\nAll philosophers have finished dining.\n");  
  
    return 0;  
}
```

OUTPUT:

```
Philosopher 1 is thinking...  
Philosopher 3 is thinking...  
Philosopher 0 is thinking...  
Philosopher 2 is thinking...  
Philosopher 4 is thinking...  
Philosopher 1 is eating...  
Philosopher 4 is eating...  
Philosopher 1 is thinking...  
Philosopher 4 is thinking...  
Philosopher 3 is eating...  
Philosopher 0 is eating...  
Philosopher 3 is thinking...  
Philosopher 2 is eating...  
Philosopher 0 is thinking...  
Philosopher 4 is eating...
```